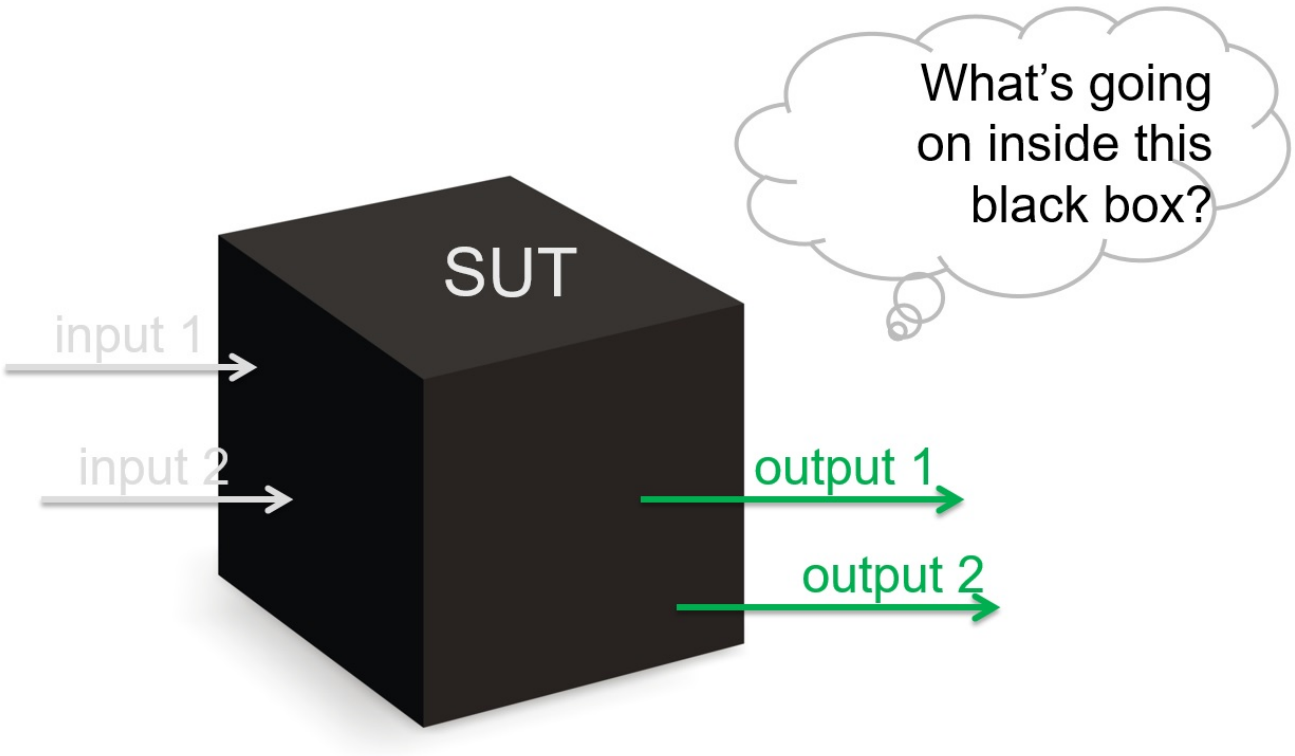
Introduction to Automata Learning

Jurriaan Rot, Radboud University

CS workshop Genoa, 9 June 2022

iCIS | Software Science
Radboud University

Problem


What's going on inside this black box?
SUT
input 1
input 2
output 1
output 2

Learning via observations

Automata learning

- constructing models through observations
- often aimed at software/hardware systems
- automata as models, representing states

Learning via observations

Automata learning

- constructing models through observations
- often aimed at software/hardware systems
- automata as models, representing states

Intriguing research topic:

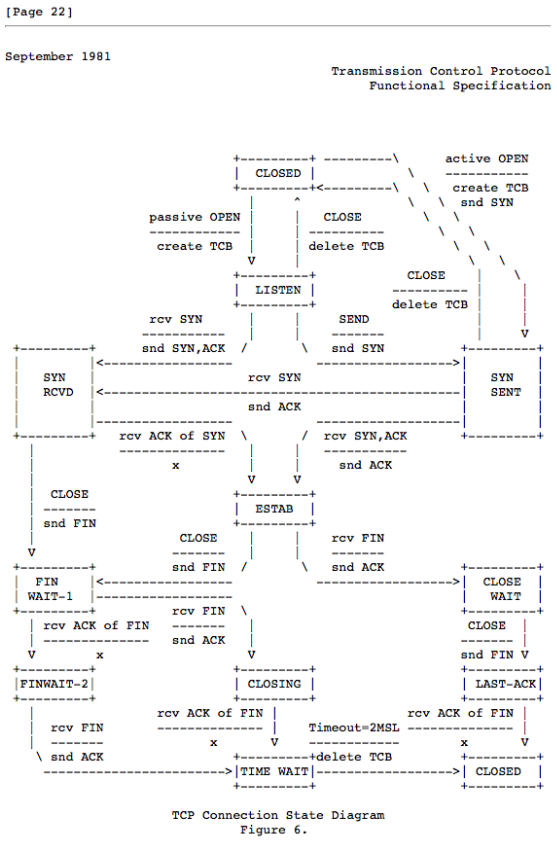
- combination of automata theory, testing, model checking
- useful technique for bug-finding
- lots of foundational questions

Overview

This talk

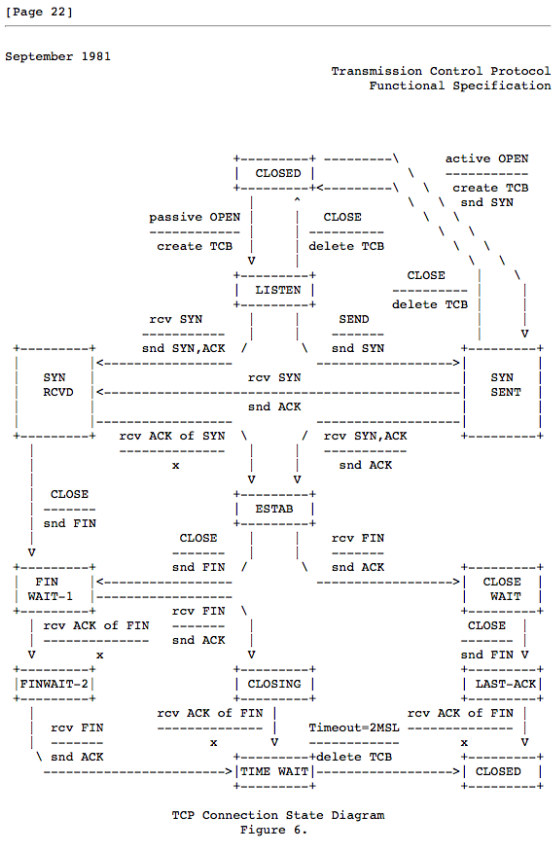
- basic introduction to active automata learning
- Angluin's classical setup and L^* algorithm
- some applications, some research directions
- recent new approach: learning via apartness and the $L^\#$ algorithm

State machine for TCP



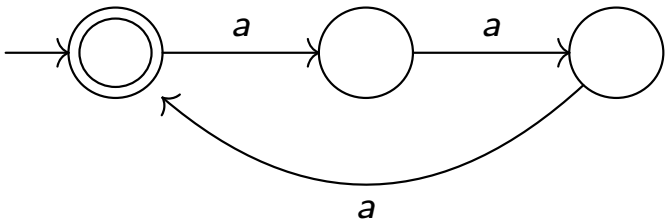
TCP Connection State Diagram
Figure 6.

State machine for TCP

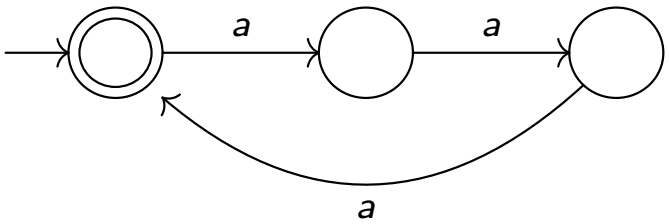


Can such a machine be learned from an implementation?

Simple language



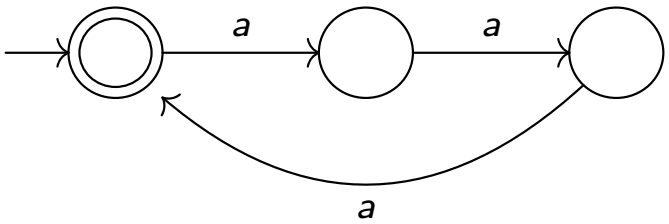
Simple language



Language $\mathcal{L}$ accepted by this automaton:

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

Simple language



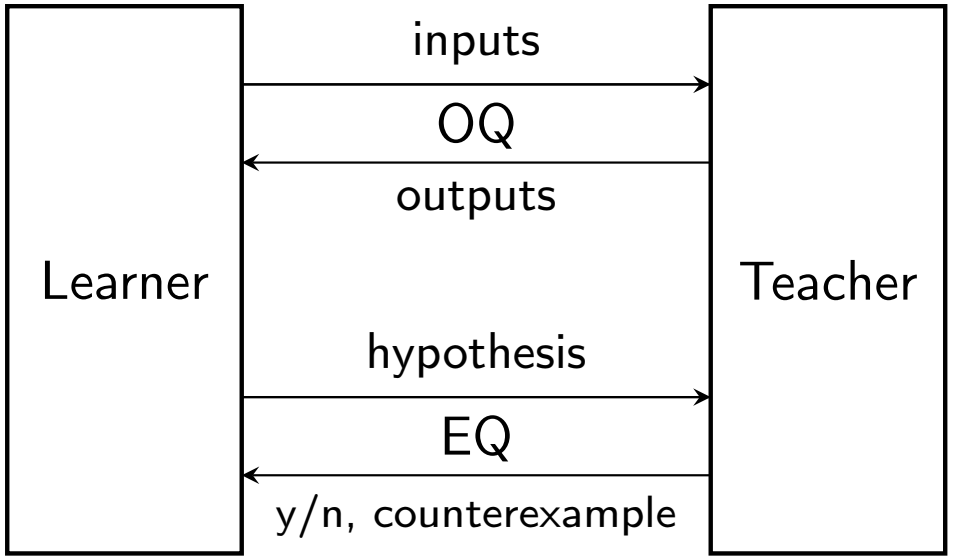
Language $\mathcal{L}$ accepted by this automaton:

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

Again, can such an automaton be learned from just "observations" about the language?

Minimally adequate teacher

Learning framework proposed by Angluin (1987):



Learner asks output queries (OQ) and equivalence queries (EQ)

Learning setup for DFAs

Finite alphabet A

Target: learn a **regular language** $\mathcal{L} \subseteq A^*$

Learning setup for DFAs

Finite alphabet A

Target: learn a **regular language** $\mathcal{L} \subseteq A^*$

Angluin's $\mathtt{L}^\star$ algorithm learns *minimal* deterministic finite automaton for $\mathcal{L}$

Learning setup for DFAs

Finite alphabet A

Target: learn a **regular language** $\mathcal{L} \subseteq A^*$

Angluin's $\mathtt{L}^\star$ algorithm learns *minimal* deterministic finite automaton for $\mathcal{L}$ assuming a *teacher/oracle* answering

- **Membership queries**

$$\boxed{w \in \mathcal{L}?}$$

Learning setup for DFAs

Finite alphabet A

Target: learn a **regular language** $\mathcal{L} \subseteq A^*$

Angluin's $\mathtt{L}^\star$ algorithm learns *minimal* deterministic finite automaton for $\mathcal{L}$ assuming a *teacher/oracle* answering

- **Membership queries**

$$\boxed{w \in \mathcal{L}?}$$

- **Equivalence queries** Given a hypothesis automaton H:

$$\boxed{L(H) = \mathcal{L}?}$$

  Negative result $\implies$ *counterexample*

$\mathtt{L}^\star$ algorithm (variation) for DFAs

$S, E \subseteq A^*$ induce a table

		ε	a
S	ε	1	0
	a	0	1
$S \cdot A$	aa	1	0

E

$\mathcal{L} = \{a^n \mid n \text{ is even}\}$

$\mathtt{L}^\star$ algorithm (variation) for DFAs

$S, E \subseteq A^*$ induce a table

$$
\begin{array}{cc|cc}
 & & \multicolumn{2}{c}{\overbrace{\qquad\qquad}^{E}} \\
 & & \varepsilon & \mathsf{a} \\
\hline
S & \varepsilon & 1 & 0 \\
 & \mathsf{a} & 0 & 1 \\
\hline
S \cdot A & \mathsf{aa} & 1 & 0
\end{array}
\qquad \mathcal{L} = \{a^n \mid n \text{ is even}\}
$$

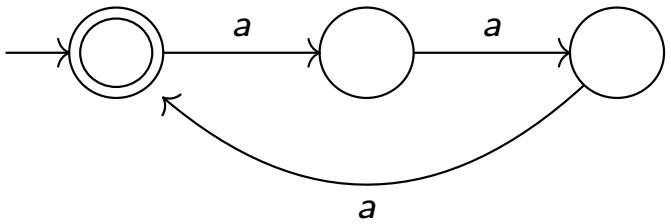
Initially $S = E = \{\varepsilon\}$

Repeat until no more counterexamples:

1. *Close* table
2. Query equivalence for *corresponding hypothesis*
3. Add suffixes of counterexamples to E

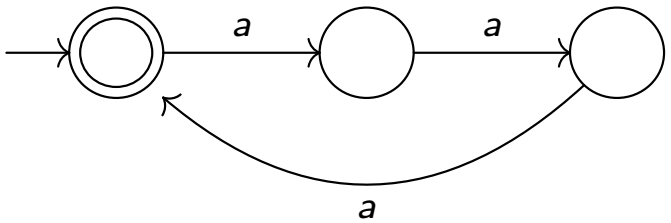
$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

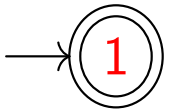


$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

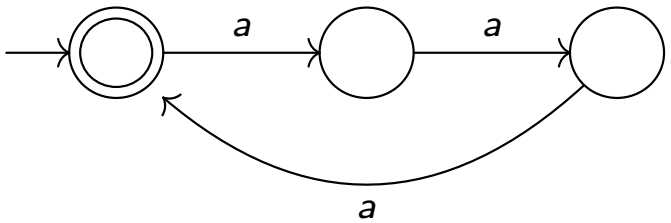


	ε
ε	1
a	0

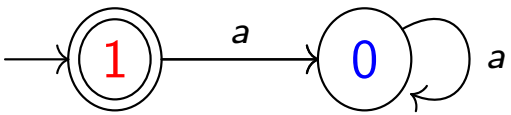


$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

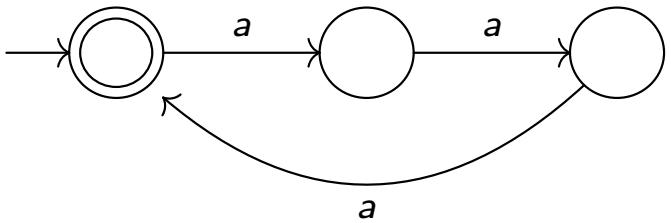


	ε
ε	1
a	0
aa	0

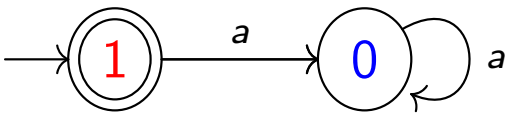


$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$



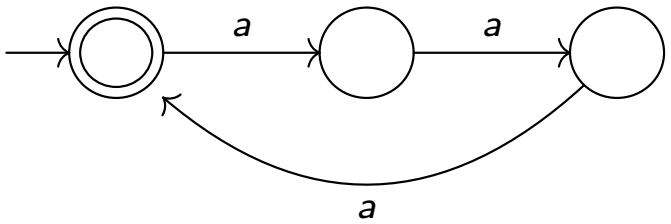
	ε
ε	1
a	0
aa	0



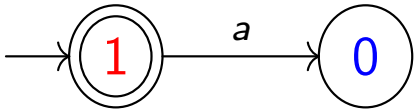
Counterexample: aaa

$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

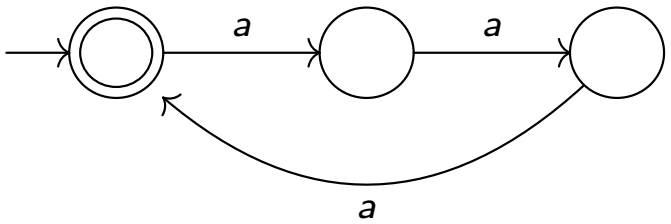


	ε	a	aa	aaa
ε	1	0	0	1
a	0	0	1	0
aa	0	1	0	0

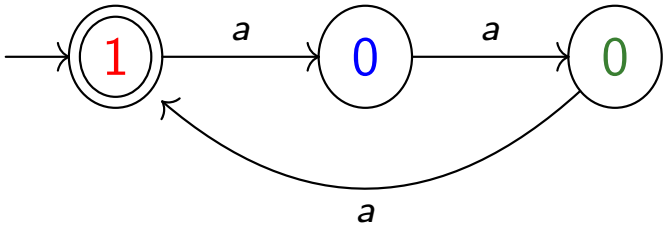


$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$

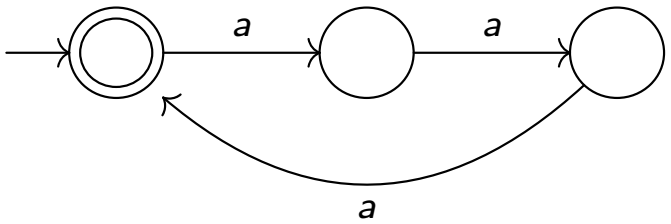


	ε	a	aa	aaa
ε	1	0	0	1
a	0	0	1	0
aa	0	1	0	0
aaa	1	0	0	1

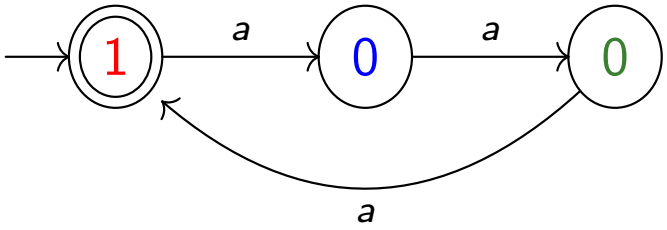


$\mathtt{L}^\star$ for DFAs, example

$$a^n \in \mathcal{L} \iff n \equiv 0 \pmod 3$$



	ε	a	aa	aaa
ε	1	0	0	1
a	0	0	1	0
aa	0	1	0	0
aaa	1	0	0	1



Correct!

Improved algorithms

Algorithm	Query Complexity	Symbol Complexity
L^*	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Maler/Pnueli	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Shabhaz	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Suffix1by1	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Rivest/Shapire	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$
Kearns/Vazirani	$\mathcal{O}(kn^2 + n^2m)$	$\mathcal{O}(kn^2m + n^2m^2)$
Observation Pack	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$
TTT	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$
ADT	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$

Here n is number of states, k is number of inputs, and m is length of maximal counterexample; table adapted from Isberner (2015).

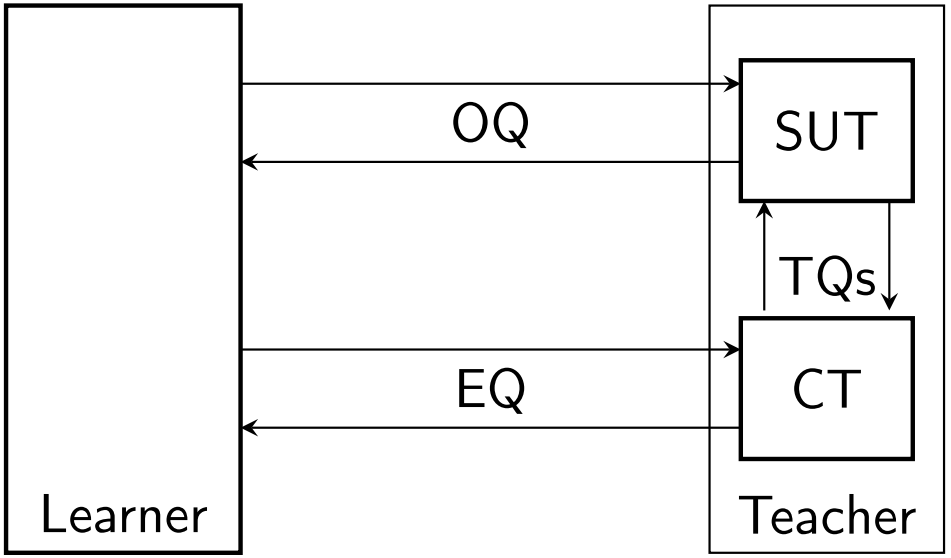
Improved algorithms

Algorithm	Query Complexity	Symbol Complexity
L^*	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Maler/Pnueli	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Shabhaz	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Suffix1by1	$\mathcal{O}(kn^2m)$	$\mathcal{O}(kn^2m^2)$
Rivest/Shapire	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$
Kearns/Vazirani	$\mathcal{O}(kn^2 + n^2m)$	$\mathcal{O}(kn^2m + n^2m^2)$
Observation Pack	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$
TTT	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$
ADT	$\mathcal{O}(kn^2 + n\log m)$	$\mathcal{O}(kn^2m + nm\log m)$

Here n is number of states, k is number of inputs, and m is length of maximal counterexample; table adapted from Isberner (2015).

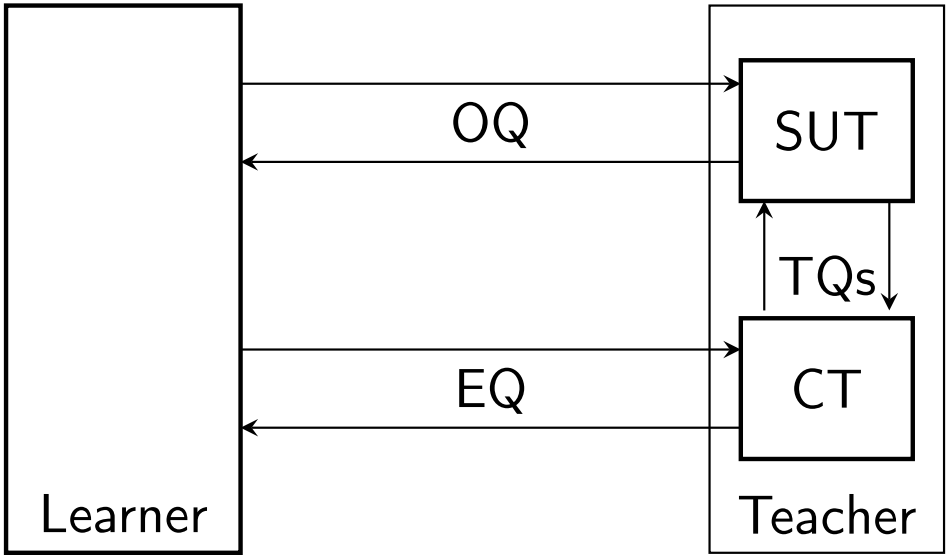
Powerful implementations available, such as learnlib and other frameworks

Black Box Checking (Peled, Vardi & Yannakakis, 2002)



Equivalence query implemented through conformance testing.

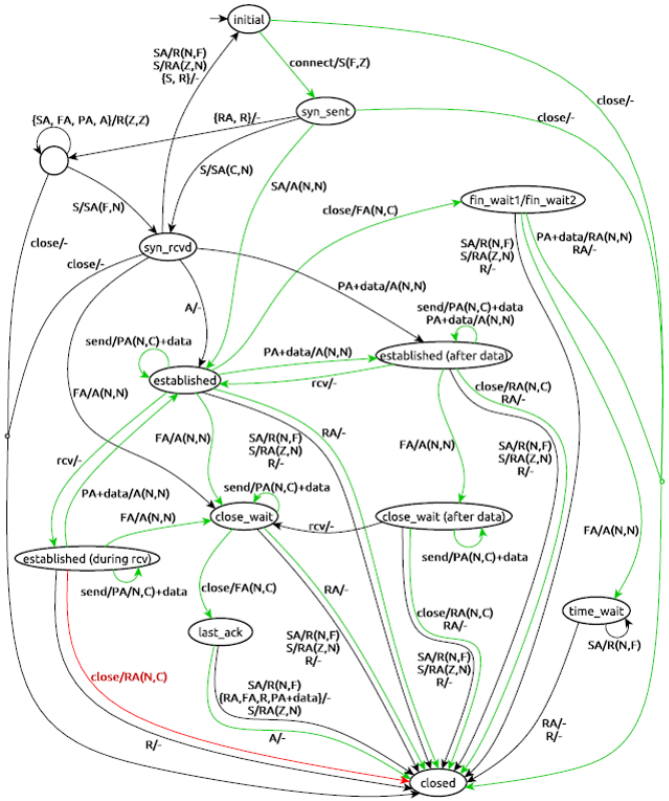
Black Box Checking (Peled, Vardi & Yannakakis, 2002)



Equivalence query implemented through conformance testing.

Further, integration with model checking: use correctness property as preprocessing step.

Bugs in Protocol Implementations



Standard violations found in implementations of major protocols:

- TLS (Usenix Security'15)
- TCP (CAV'16)
- SSH (Spin'17)

Bugs in Protocol Implementations

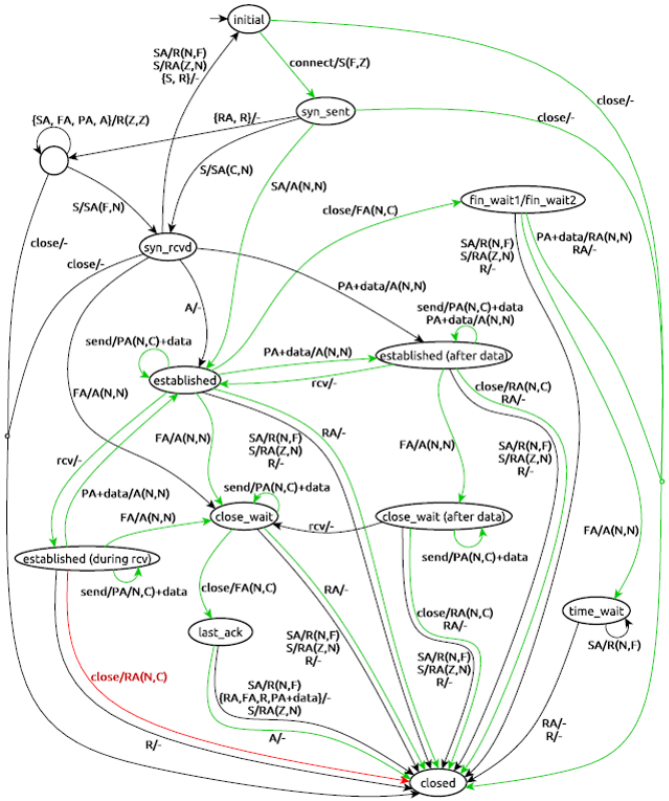


Figure: learned model of TCP client (Fiterau-Brostean et al., CAV '16)

Standard violations found in implementations of major protocols:

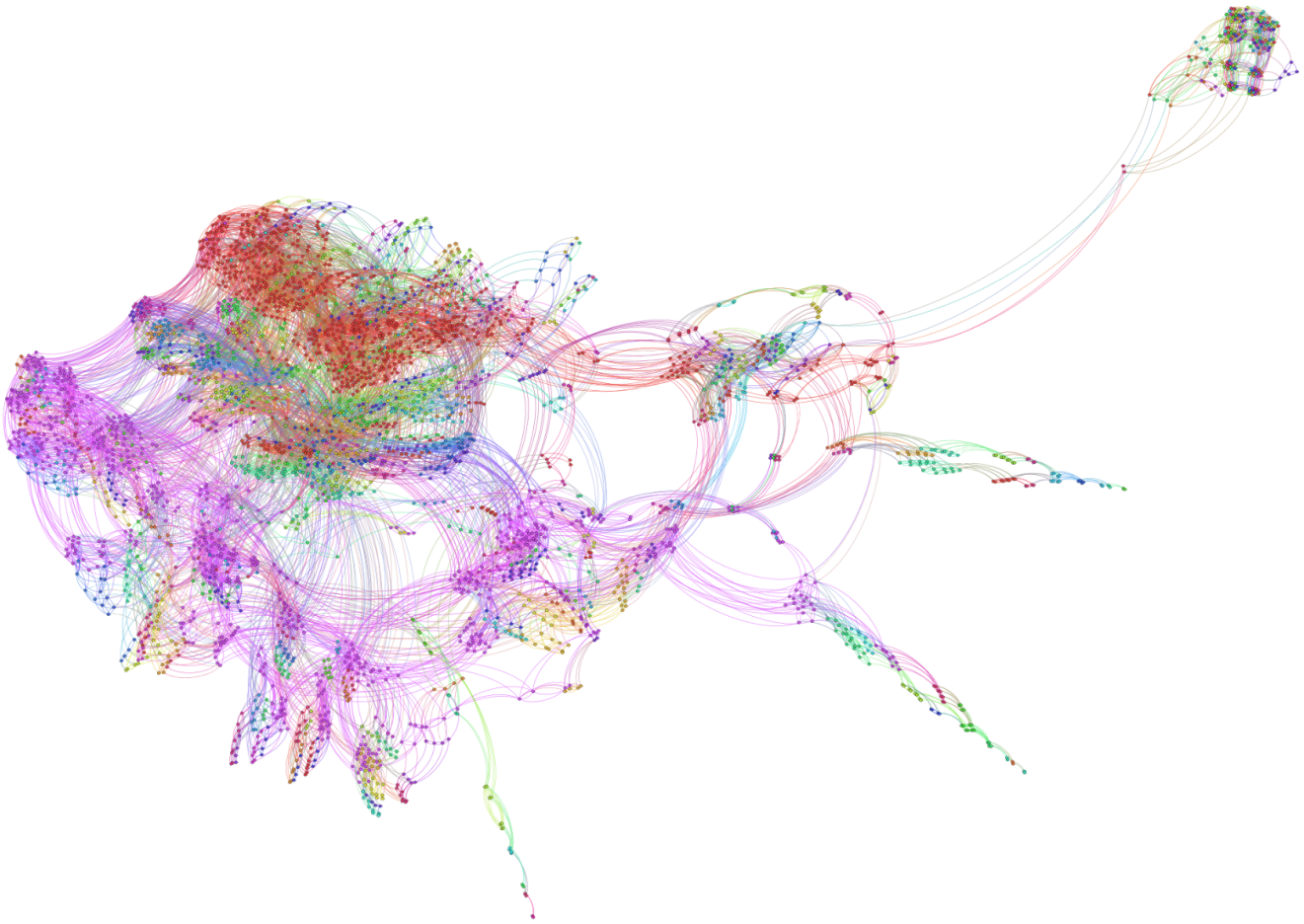
- TLS (Usenix Security'15)
- TCP (CAV'16)
- SSH (Spin'17)

These findings led to bug fixes in implementations.

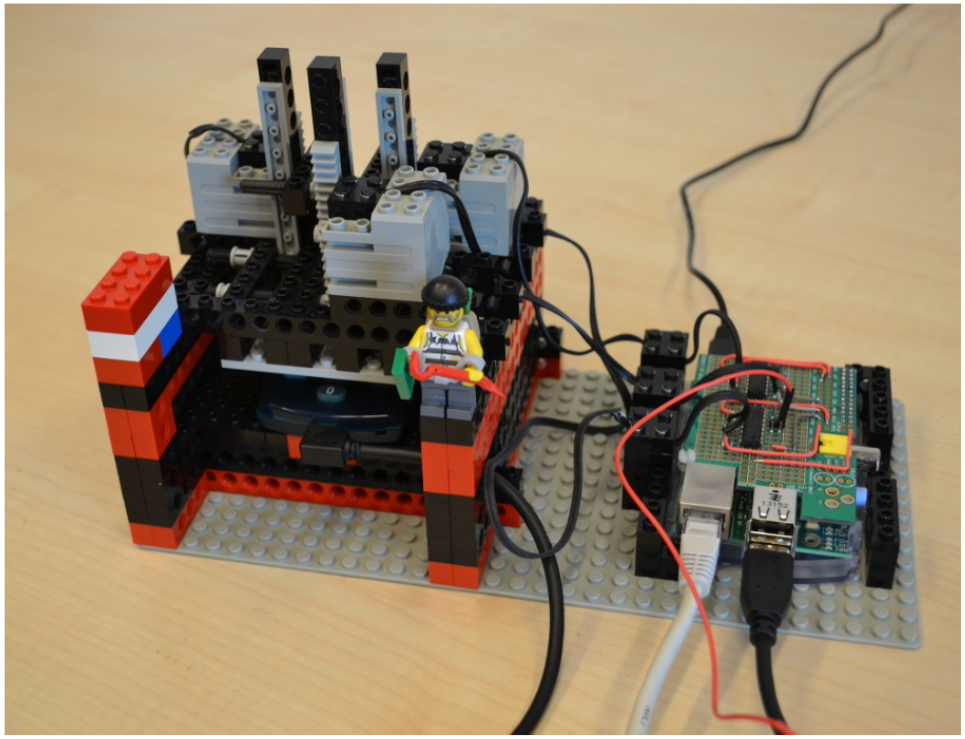
Engine Status Manager in Océ Printer (ICFEM'15)

Can we learn interface models of realistic printer controllers?

Mealy Machine for Engine Status Manager



Bank cards



(Chalupar et al, WOOT 2014)

Research in active learning

Since Angluin's early work, lots of development and also still many open problems.

Research in active learning

Since Angluin's early work, lots of development and also still many open problems.

- case studies — many, many examples
- scope — extensions to many other types of automata, categorical generalisation, etc.
- efficiency of algorithms — in particular reducing the number of queries made.

Research in active learning

Since Angluin's early work, lots of development and also still many open problems.

- case studies — many, many examples
- scope — extensions to many other types of automata, categorical generalisation, etc.
- efficiency of algorithms — in particular reducing the number of queries made.

Further topics: compositionality, quality of hypotheses, ...

New perspective: $L^{\#}$

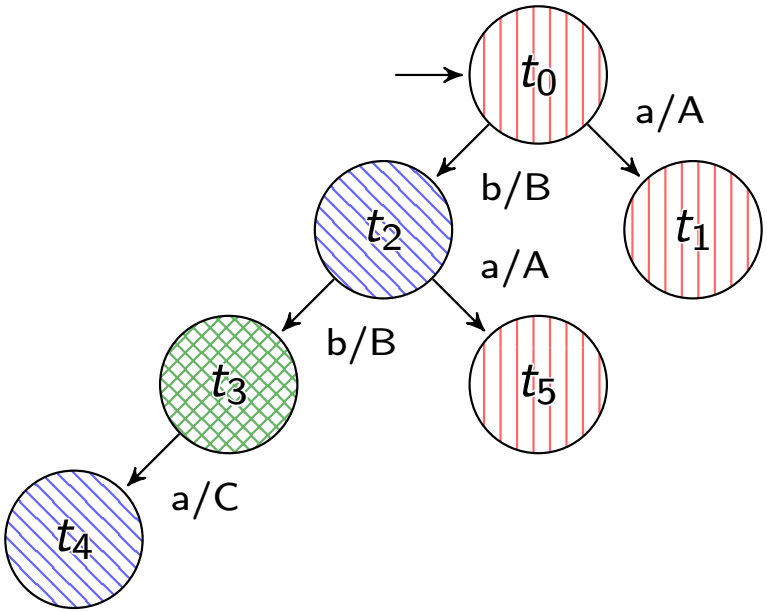
Inspired by Brouwer's notion of apartness and Dijkstra's shortest path algorithm

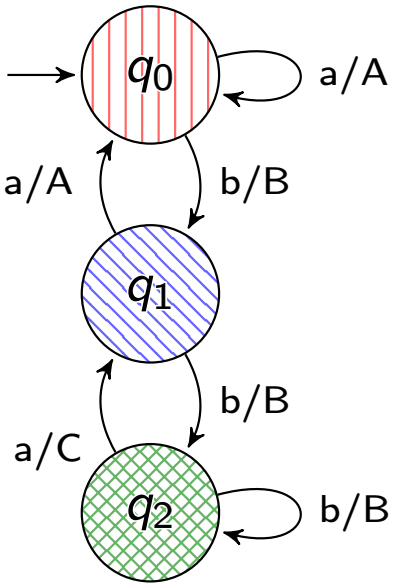
Joint work Frits Vaandrager, Bharat Garhewal, Thorsten Wissman (TACAS '22)

Observation Trees

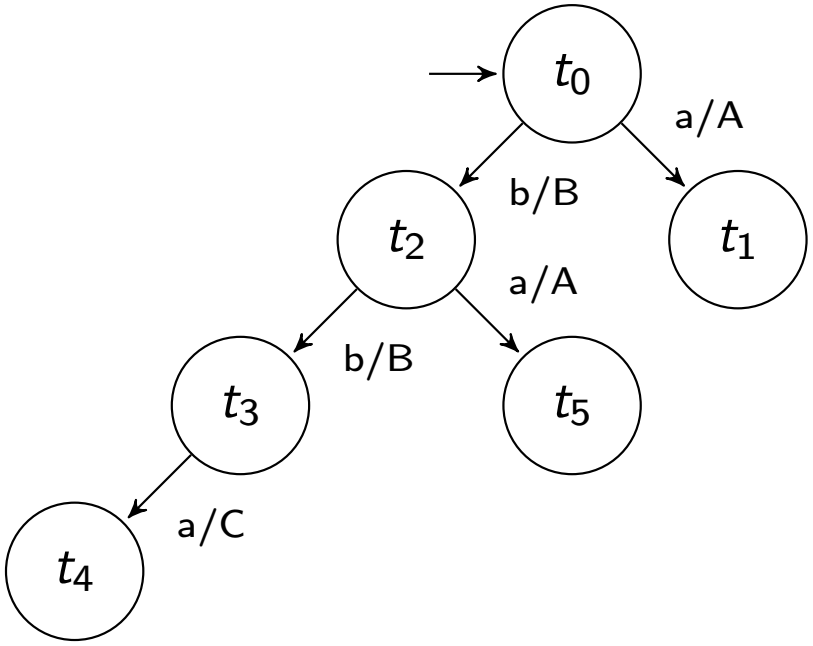
$L^{\#}$ learns Mealy machines.

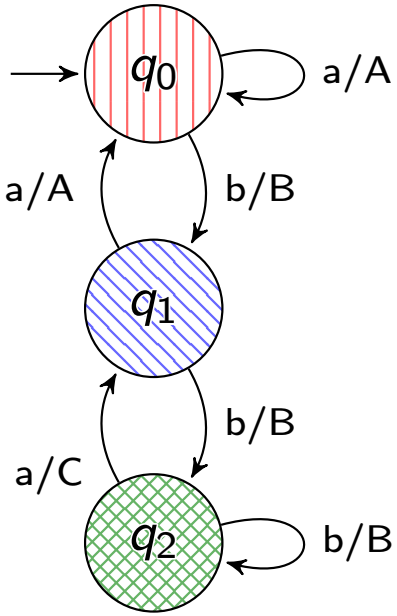
Central data structure is not a table, but an observation tree that stores queries:





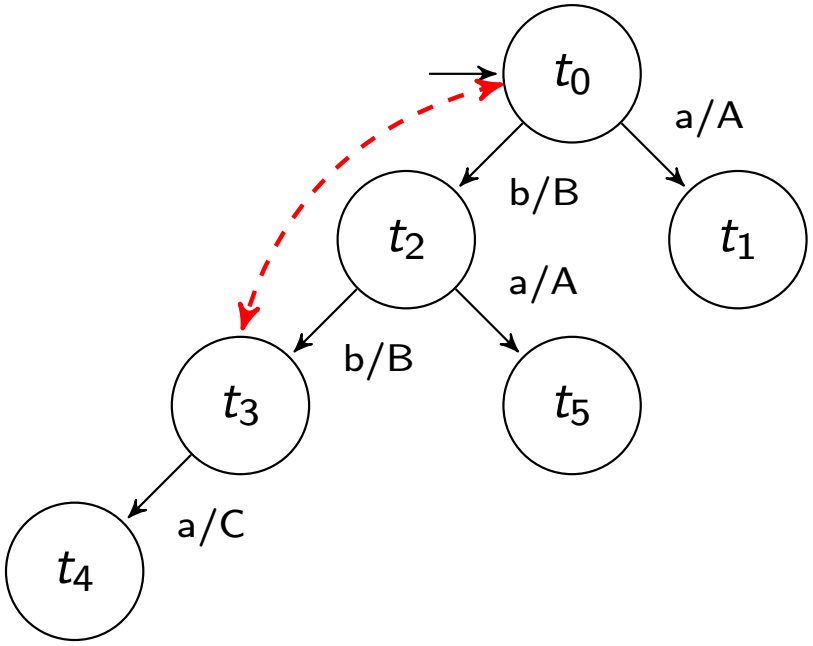
Apartness



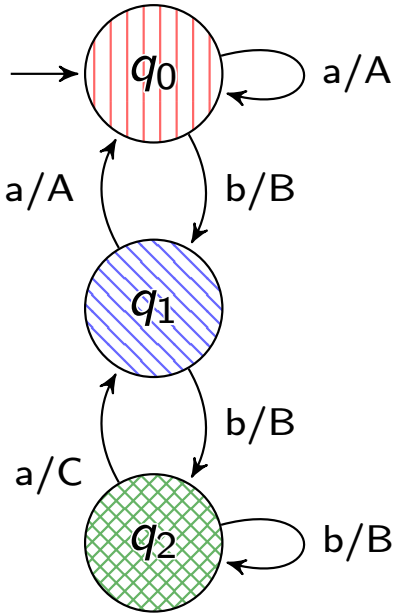


Apartness: Two states t, u in an observation tree are apart, $t \# u$, if they both enable an input sequence $\sigma \in I^*$ for which the outputs are different.

Apartness

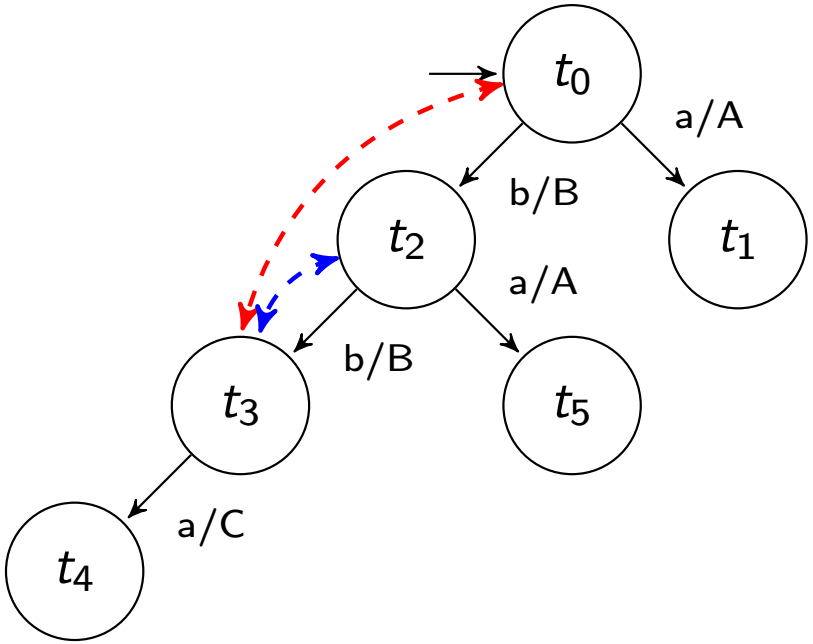


$a \vdash t_3 \# t_0$



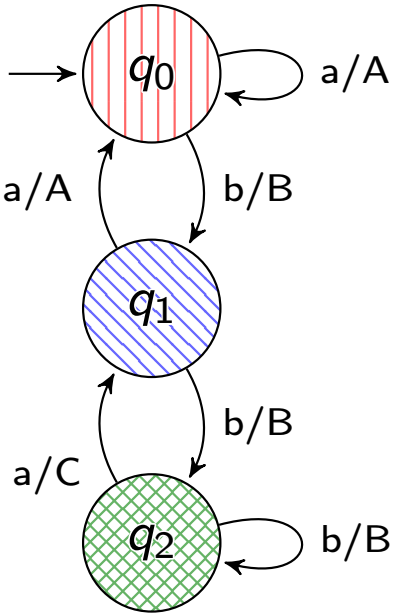
Apartness: Two states t, u in an observation tree are apart, $t \# u$, if they both enable an input sequence $\sigma \in I^*$ for which the outputs are different.

Apartness



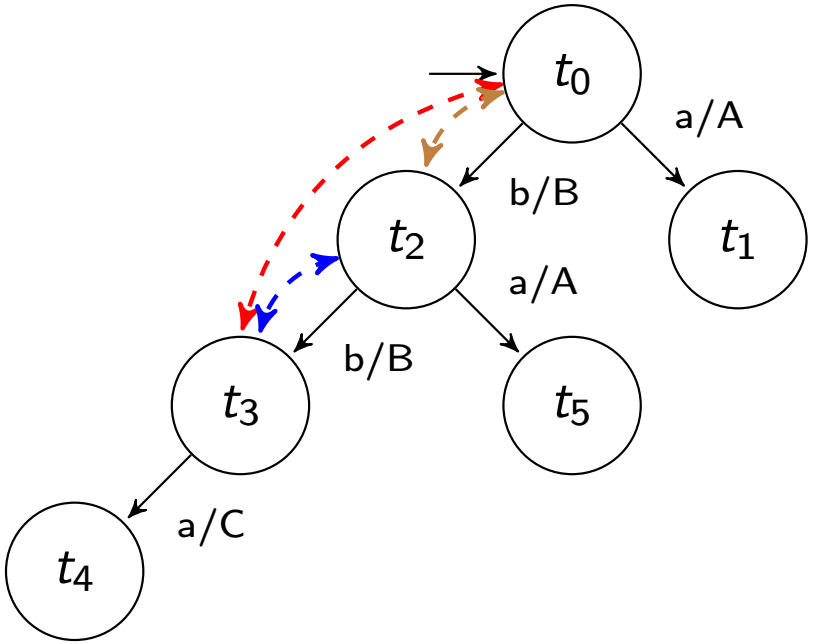
$a \vdash t_3 \# t_0$

$a \vdash t_3 \# t_2$



Apartness: Two states t, u in an observation tree are apart, $t \# u$, if they both enable an input sequence $\sigma \in I^*$ for which the outputs are different.

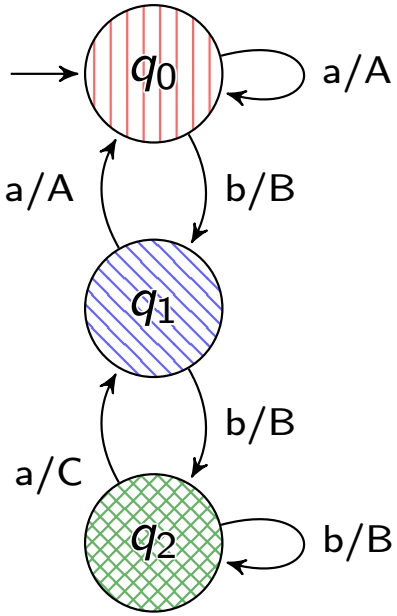
Apartness



$a \vdash t_3 \# t_0$

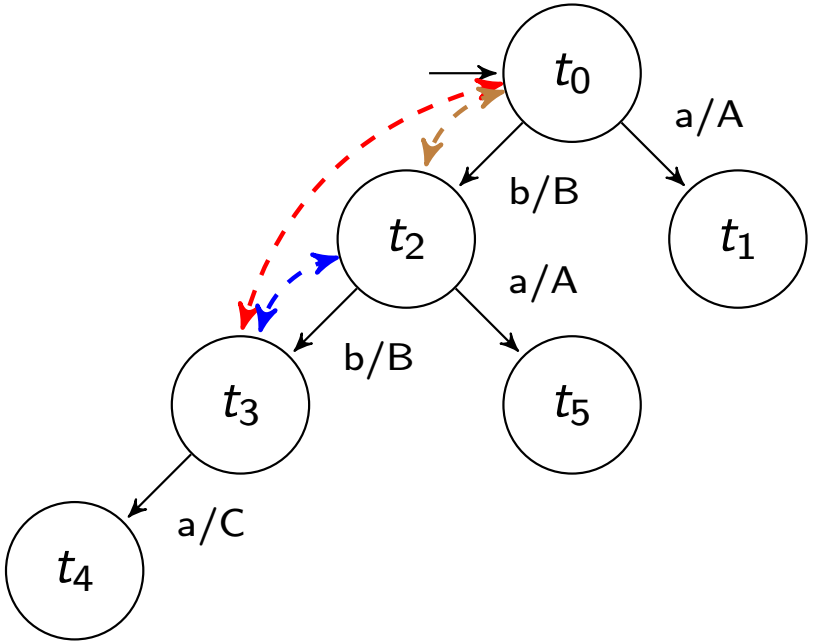
$a \vdash t_3 \# t_2$

$ba \vdash t_2 \# t_0$



Apartness: Two states t, u in an observation tree are apart, $t \# u$, if they both enable an input sequence $\sigma \in I^*$ for which the outputs are different.

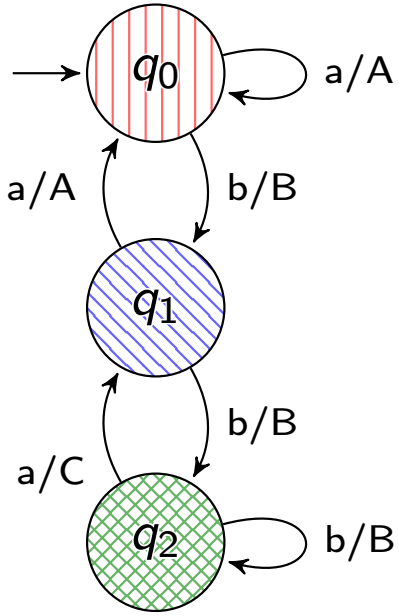
Apartness



$a \vdash t_3 \# t_0$

$a \vdash t_3 \# t_2$

$ba \vdash t_2 \# t_0$



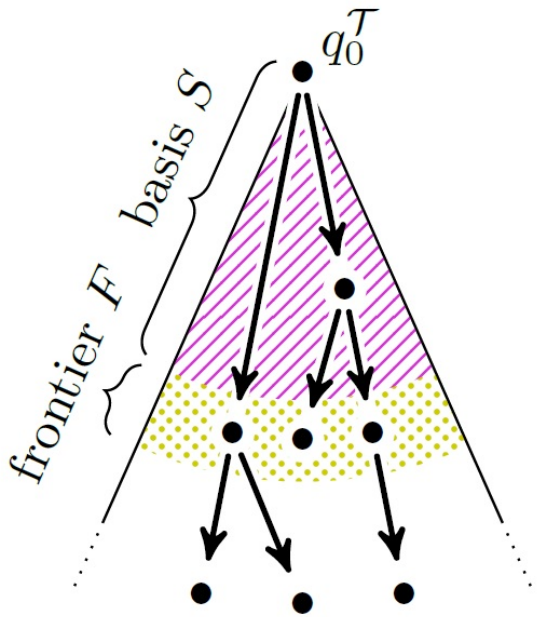
Apartness: Two states t, u in an observation tree are apart, $t \# u$, if they both enable an input sequence $\sigma \in I^*$ for which the outputs are different.

Whenever states are apart in the observation tree, the learner knows these correspond to distinct states in the hidden Mealy machine.

Basis and Frontier

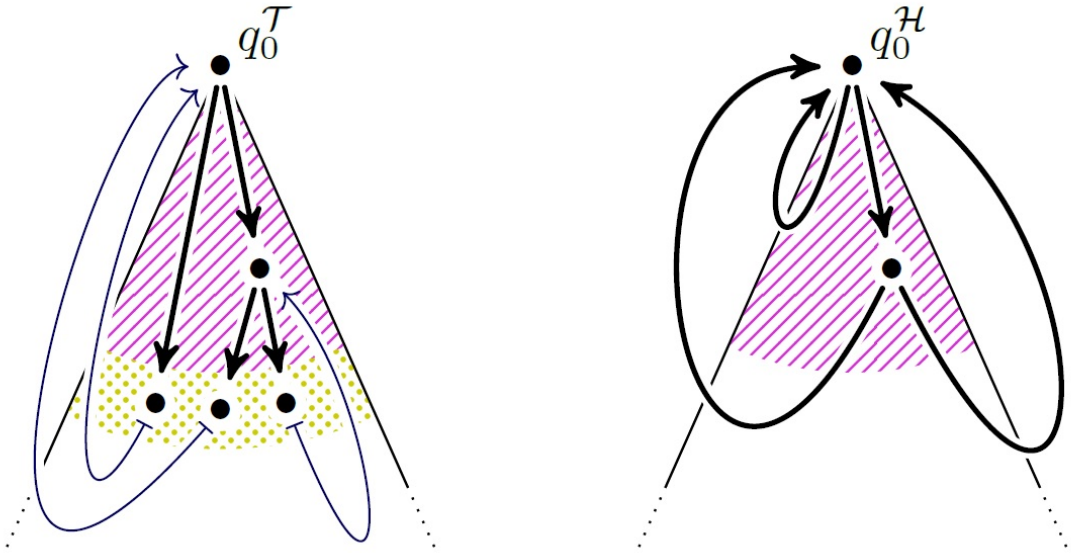
Like the shortest path algorithm of Dijkstra (1959), $L^{\#}$ subdivides the states of the observation tree into three sets:

1. basis S: these states form a subtree of the observation tree and are pairwise apart
2. frontier F: the immediate non-basis successors of basis states; the next node to be added to S will be selected from F
3. the remaining states



Hypothesis Construction

Construct a hypothesis whenever each state in the frontier is apart from all but one states in the basis: it is identified.



$L^{\#}$: further comments

- the algorithm has the same (query) complexity as the state-of-the-art
- experimental evaluation shows it's competitive
- but it's relatively simple!

Conclusion

Model learning is an exciting topic of research

Conclusion

Model learning is an exciting topic of research

Thanks for you attention!